

Compiler Design DSL Project

9.0 Design Assessment

Project Description

This project focuses on the design and implementation of a simple Domain Specific Language (DSL) compiler using Flex and Bison tools. The compiler performs the major phases of compilation including lexical analysis, syntax parsing, Abstract Syntax Tree (AST) construction, semantic analysis, and intermediate code generation.

The system accepts arithmetic assignment statements as input and verifies whether the syntax follows the grammar rules defined in the parser. Flex is used to identify tokens such as identifiers, numbers, operators, and delimiters, while Bison is used to validate the syntax structure of the input statements.

After successful parsing, the compiler simulates AST creation and performs semantic analysis to ensure the correctness of expressions. Intermediate code generation is also implemented to represent arithmetic operations in simplified three-address code format.

The project demonstrates the basic workflow of compiler construction and provides practical understanding of how programming language statements are translated and analyzed internally. The compiler was developed using C language in Visual Studio Code and uploaded to GitHub with proper project structure and documentation.

Objective

The main objective of this project is to understand the internal working of a compiler and implement the major stages of compilation using industry-standard compiler tools such as Flex and Bison.

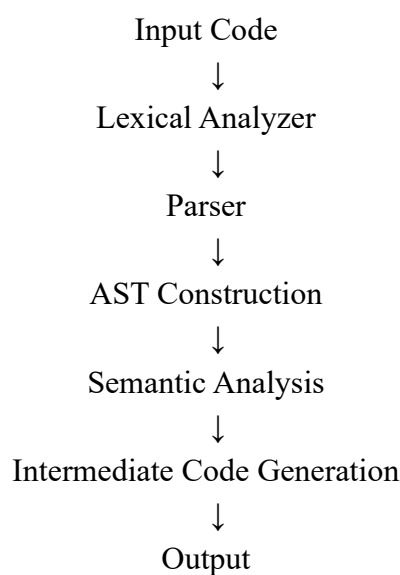
The project aims to:

- Perform lexical analysis on DSL statements
- Parse input statements using grammar rules
- Construct syntax structures
- Perform semantic validation
- Generate intermediate code representation

Tools and Technologies Used

Tool	Purpose
Flex	- Lexical Analysis
Bison	- Syntax Parsing
GCC	- Compilation
VS Code	- Code Development
GitHub	- Version Control and Project Hosting

System flow



Module Explanation

1. Lexical Analysis

Lexical analysis is the first phase of the compiler. In this stage, the input source code is scanned and converted into tokens. Flex was used to identify identifiers, numbers, arithmetic operators, assignment operators, and delimiters.

Example tokens:

- IDENTIFIER
- NUMBER
- PLUS
- ASSIGN
- SEMICOLON

2. Parsing

Parsing is the second phase where the token sequence is checked against grammar rules. Bison was used to define grammar productions for assignment statements and arithmetic expressions.

Example supported statement:

`a = 4 + 5;`

3. AST Construction

Abstract Syntax Tree (AST) construction represents the hierarchical structure of expressions and statements. In this project, AST generation was simulated through parser actions to demonstrate expression structure creation.

AST helps in later compiler phases such as optimization and code generation.

4. Semantic Analysis

Semantic analysis verifies the correctness of statements beyond syntax. It ensures meaningful operations are performed in the program.

The project performs basic semantic validation and displays a success message after verifying valid expressions.

5. Intermediate Code Generation

Intermediate code generation produces a simplified low-level representation of expressions.

For the input:

```
a = 4 + 5;
```

Generated intermediate code:

```
t1 = 4 + 5
```

```
a = t1
```

This representation is useful for optimization and machine code generation.

Sample Input

```
a = 4 + 5;
```

Sample Output

Valid Statement

AST Created

Semantic Analysis Successful

```
t1 = 4 + 5
```

```
a = t1
```

```
1 %{
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 void yyerror(const char *s);
6 int yylex();
7 %}
8
9 %token NUMBER IDENTIFIER PRINT
10 %token PLUS ASSIGN SEMICOLON
11
12 %%
13
14 program:
15     statement
16     ;
17
18 statement:
19     IDENTIFIER ASSIGN expression SEMICOLON
20     ;
21
22 expression:
23     NUMBER
24     | NUMBER PLUS NUMBER
25     ;
26
27 %%
28
29 void yyerror(const char *s) {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- lakshadeepak@LAKSHADEEPAKS-MacBook-Air src % cd ~/Desktop/Compiler-Design-DSL/src
- lakshadeepak@LAKSHADEEPAKS-MacBook-Air src % bison -d parser.y
- lakshadeepak@LAKSHADEEPAKS-MacBook-Air src % flex lexer.l
- lakshadeepak@LAKSHADEEPAKS-MacBook-Air src % gcc lex.yy.c parser.tab.c -o compiler
- lakshadeepak@LAKSHADEEPAKS-MacBook-Air src % ./compiler
- Enter DSL code:
- a = 4 + 5; *C
- Valid Statement
- lakshadeepak@LAKSHADEEPAKS-MacBook-Air src % ./compiler
- Enter DSL code:
- a = 3 + 7;
- Valid Statement
- lakshadeepak@LAKSHADEEPAKS-MacBook-Air src %

Tip: Try the Plan agent to research and plan before implementing changes.

+ : parser.y

Describe what to b...

+ ? ... ↑

Local ... D ...

conclusion

This project successfully implemented a simple DSL compiler using Flex and Bison. The compiler performs lexical analysis, parsing, AST construction, semantic checking, and intermediate code generation. The project helped in understanding the core concepts of compiler design and demonstrated how compiler phases work together to process programming language statements.

The project can be further extended to support variables, loops, conditional statements, optimization techniques, and machine code generation.